

G.H. Raisoni College of Engineering and Management

Department of Information Technology

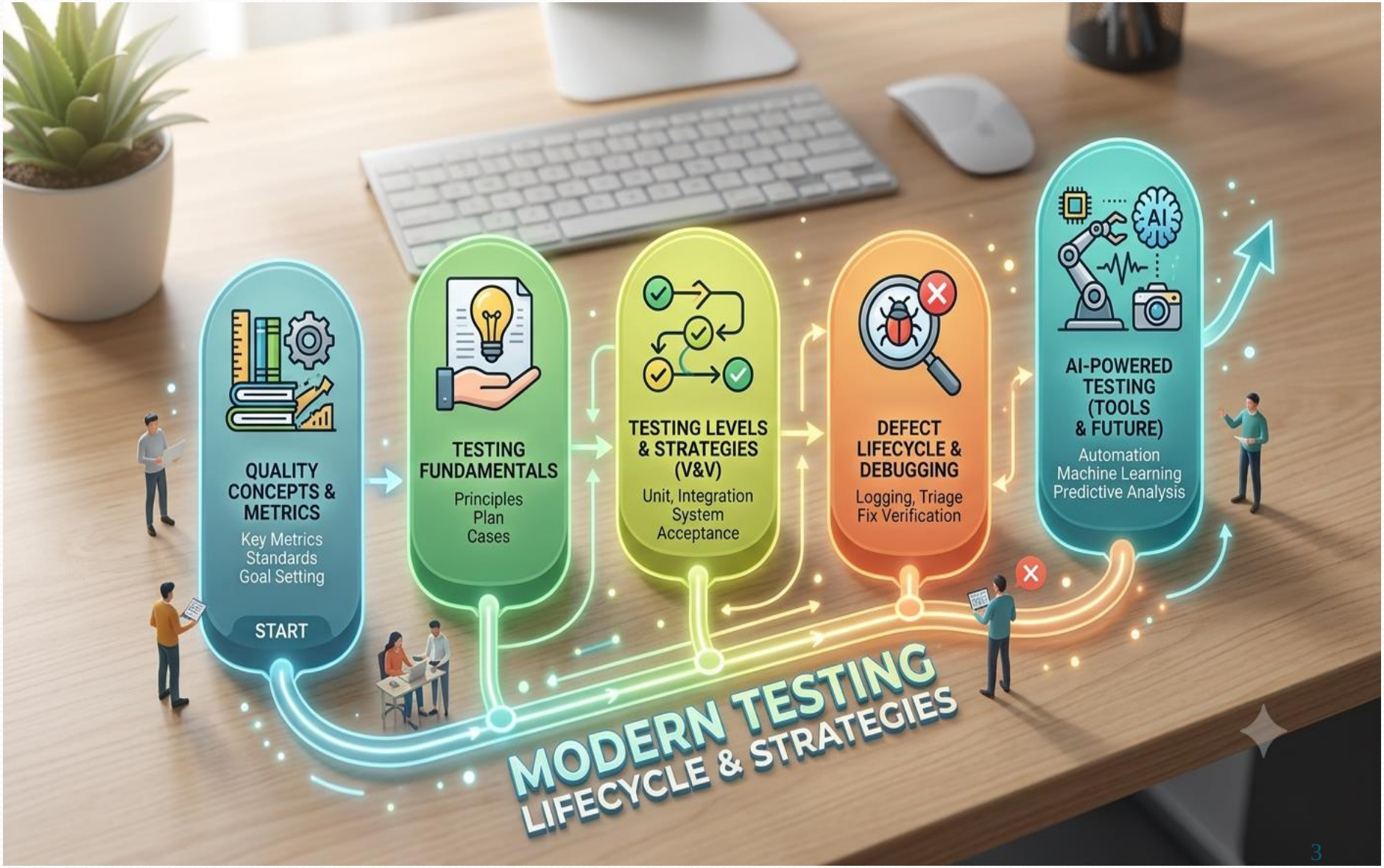
Unit 4 : SOFTWARE TESTING & QUALITY ASSURANCE

Prof. Sagar Mahajan

We don't just learn testing; we learn how to build a fortress of quality. Let's go!



Agenda & RoadMap



What is Quality?

- **Definition**: *Fitness for Purpose*. Meeting explicit + implicit user needs.
- **Explicit:** What user *said* they want.
- **Implicit:** What user *actually needs* (Security, Speed, Usability).



Software Quality Matrices?

- *"You can't improve what you don't measure.*
- **Defect Density** = Total Defects / KLOC (Kilo Lines of Code).

  Target: < 1/KLOC.
- **Mean Time To Failure (MTTF)** = Average time until the software crashes.
- **Code Coverage %** = How much of your code is actually tested?



Software Quality Matrices?

Metric	Formula	Unit
Defect Density	Total Defects ÷ KLOC (Kilo Lines of Code)	Defects per KLOC
Mean Time to Failure (MTTF)	Total Operating Time ÷ Number of Failures	Hours (or Minutes)
Code Coverage %	(Lines executed during test ÷ Total lines of code) × 100	Percentage (%)

Numerical on Defect Density :

Goal: To check how many bugs are hiding in every 1000 lines of code.

- A module has **2 KLOC** (2000 lines). Testers found **10 defects**. What is the Defect Density?

Solution: 10 ÷ 2 = **5 Defects/KLOC**.

Meaning: For every 1000 lines, 5 bugs. Decent, can improve.

Numerical on Defect Density :

- A banking module has **5 KLOC** and **25 defects**. Calculate density.
- A small login page has just **0.4 KLOC** (400 lines) but contains **8 defects**.
- An e-commerce cart has **8 KLOC** with **40 defects**. Find density.
- A new microservice has **3 KLOC** and only **3 defects**

Numerical on MTTF :

Goal: To find out how long (on average) the software runs smoothly before it crashes.

- An ATM software runs for **100 hours** total, but crashes **4 times**. Find MTTF.

- **Solution:** 100 ÷ 4 = **25 Hours**.

Meaning: The ATM crashes every 25 hours. That means daily maintenance. Not great for banks.

Numerical on MTTF :

- A smartwatch app runs for **50 hours** and fails **10 times**.
- A hospital patient monitor runs for **200 hours** and fails **8 times**.
- A gaming console runs for **30 hours** and hangs/crashes **15 times**.
- A cloud server runs for **72 hours** and fails **6 times**.

Numerical on Code Coverage (%) :

Goal: To see how much of your written code is actually being tested by your test cases.

- Your total code is **1000 lines**. Your test suite executed **800 lines**. Find coverage %.

- **Solution:** (800 ÷ 1000) × 100 = **80%**.

Legendary Meaning: 80% is the *minimum* industry standard. You pass, but 20% is untested (danger zone).

Numerical on Code Coverage (%) :

- Total code is **500 lines**. Tests executed **450 lines**.
- Total code is **2000 lines**. Tests executed **1200 lines**.
- Total code is **1000 lines**. Tests executed **950 lines**.
- Total code is **500 lines**. Tests executed just **250 lines**.

Metrics :

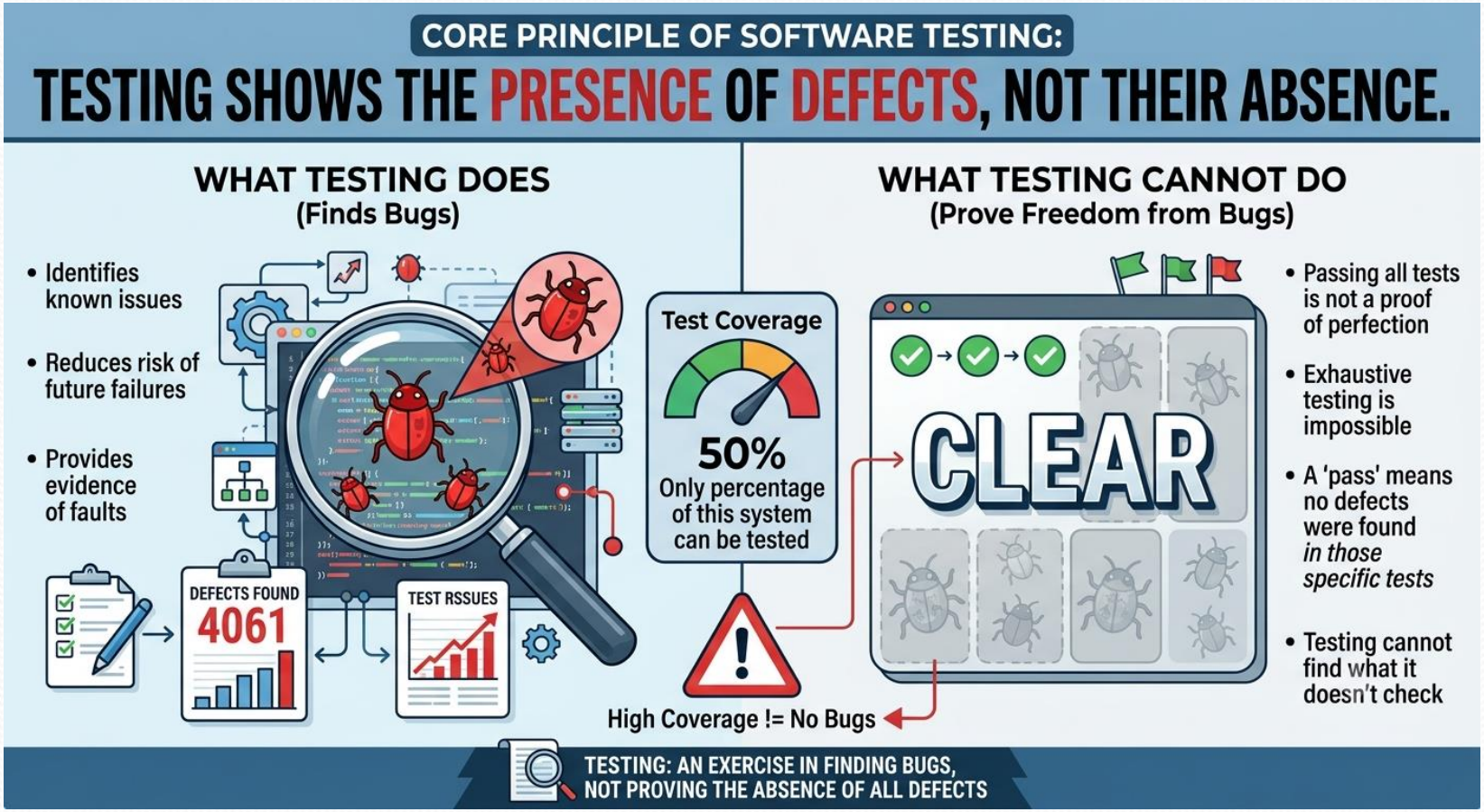
Metric	Poor Score	Good Score	Elite Score
Defect Density	> 10/KLOC	2 - 5/KLOC	< 1/KLOC
MTTF	< 10 Hours	50 - 100 Hours	> 500 Hours
Code Coverage	< 70%	80% - 85%	> 90%

Testing Principles:

- **Principle 1:** Testing shows the presence of defects, NOT their absence.
- **Principle 2:** Exhaustive Testing is Impossible.
- **Principle 3:** Early Testing saves Time and Money. (Shift-Left)
- **Principle 4:** Defects Cluster Together. (The 80/20 Rule)
- **Principle 5:** Beware of the Pesticide Paradox.
- **Principle 6**:Testing is Context Dependent.
- **Principle 7:** Absence-of-Errors Fallacy.

Testing Principles:

- **Principle 1:** Testing shows the presence of defects, NOT their absence.



Testing Principles:

- **Principle 1:** Testing shows the presence of defects, NOT their absence.

Example : You run 10,000 test cases and they all PASS. This does NOT mean the software has 0 bugs. It means you haven't *found* them yet. The code might crash in production with a rare data combination.

Testing Principles:

- **Principle 2:** Exhaustive Testing is Impossible.



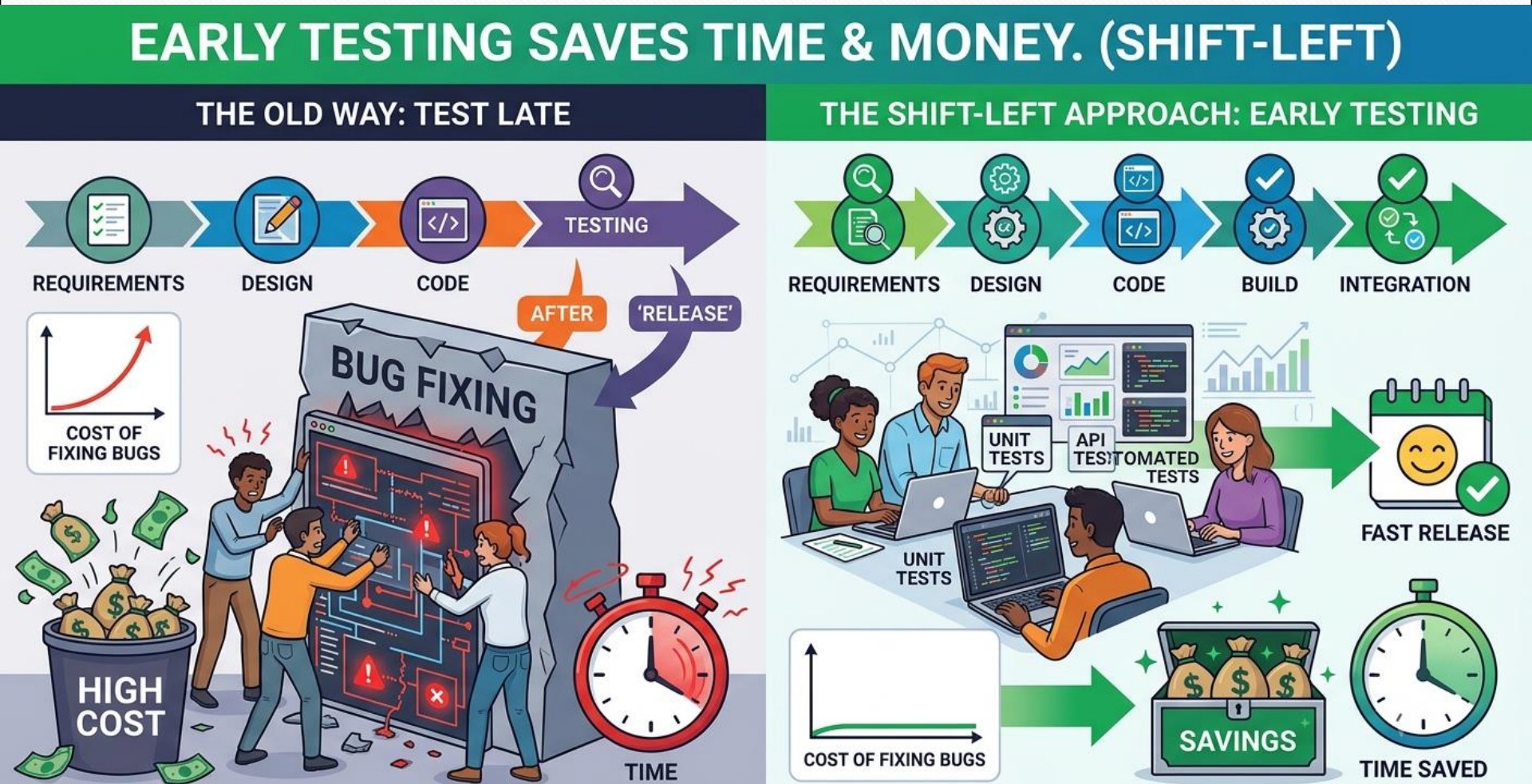
Testing Principles:

- **Principle 2:** Exhaustive Testing is Impossible.

Example : A simple login field accepting an email has billions of possible string combinations. You cannot test every single one. You use **Equivalence Partitioning** (valid/invalid groups) and **Boundary Value Analysis** (test min/max lengths).

Testing Principles:

- **Principle 3:** Early Testing saves Time and Money. (Shift-Left)



Testing Principles:

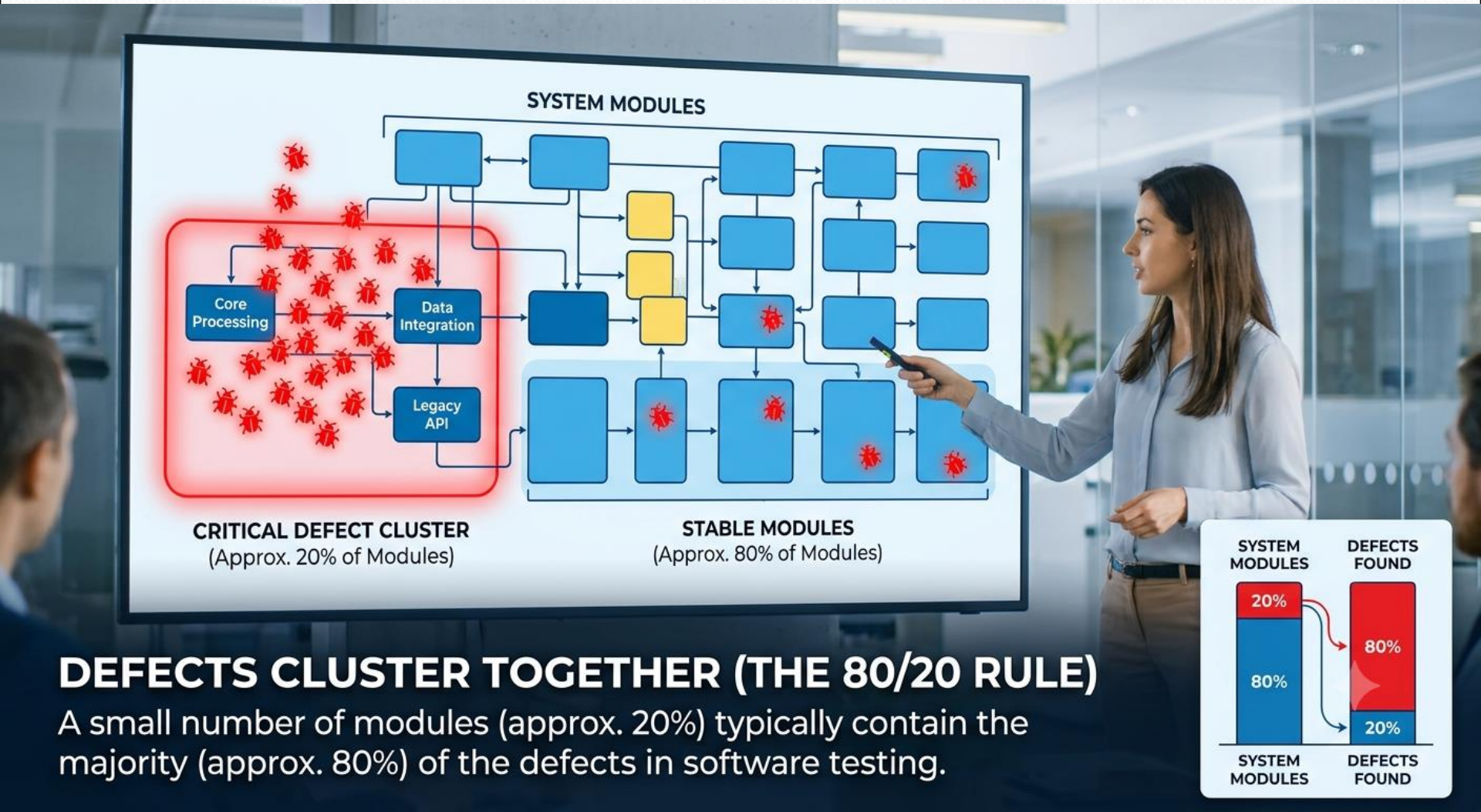
- **Principle 3:** Early Testing saves Time and Money. (Shift-Left)

Example : If you find a design flaw (e.g., wrong database schema) during the *Requirements* phase, it costs **$1** to fix. If you find it in Production, it costs $1000+** (hotfixes, data migration, downtime).

Start testing from Day 1 of coding (Unit Tests).

Testing Principles:

- **Principle 4:** Defects Cluster Together. (The 80/20 Rule)



Testing Principles:

- **Principle 4:** Defects Cluster Together. (The 80/20 Rule)

Example : In most projects, **80% of critical bugs** are found in **20% of the modules** (usually the most complex business logic or frequently changed code). Use **Risk-Based Testing** – focus your regression tests on these hot zones.

Testing Principles:

- **Principle 5:** Beware of the Pesticide Paradox.



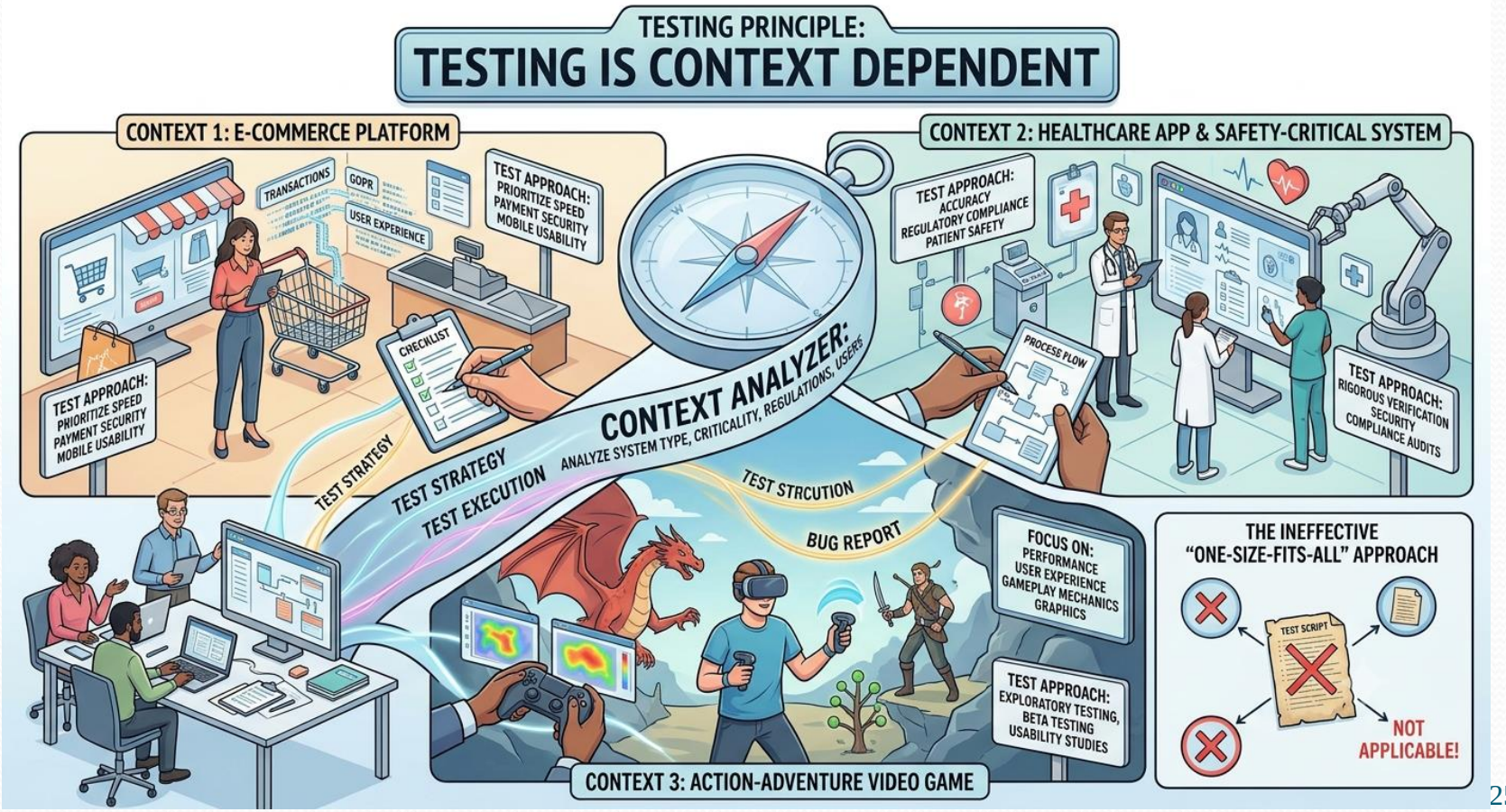
Testing Principles:

- **Principle 5:** Beware of the Pesticide Paradox**.**

Example : If you run the same Selenium regression suite for 6 months without updating it, the bugs mutate! The code changes, new bugs appear in new areas, but your old scripts will pass. You must **constantly review and update** your test cases to find new defects.

Testing Principles:

- **Principle 6**:Testing is Context Dependent.



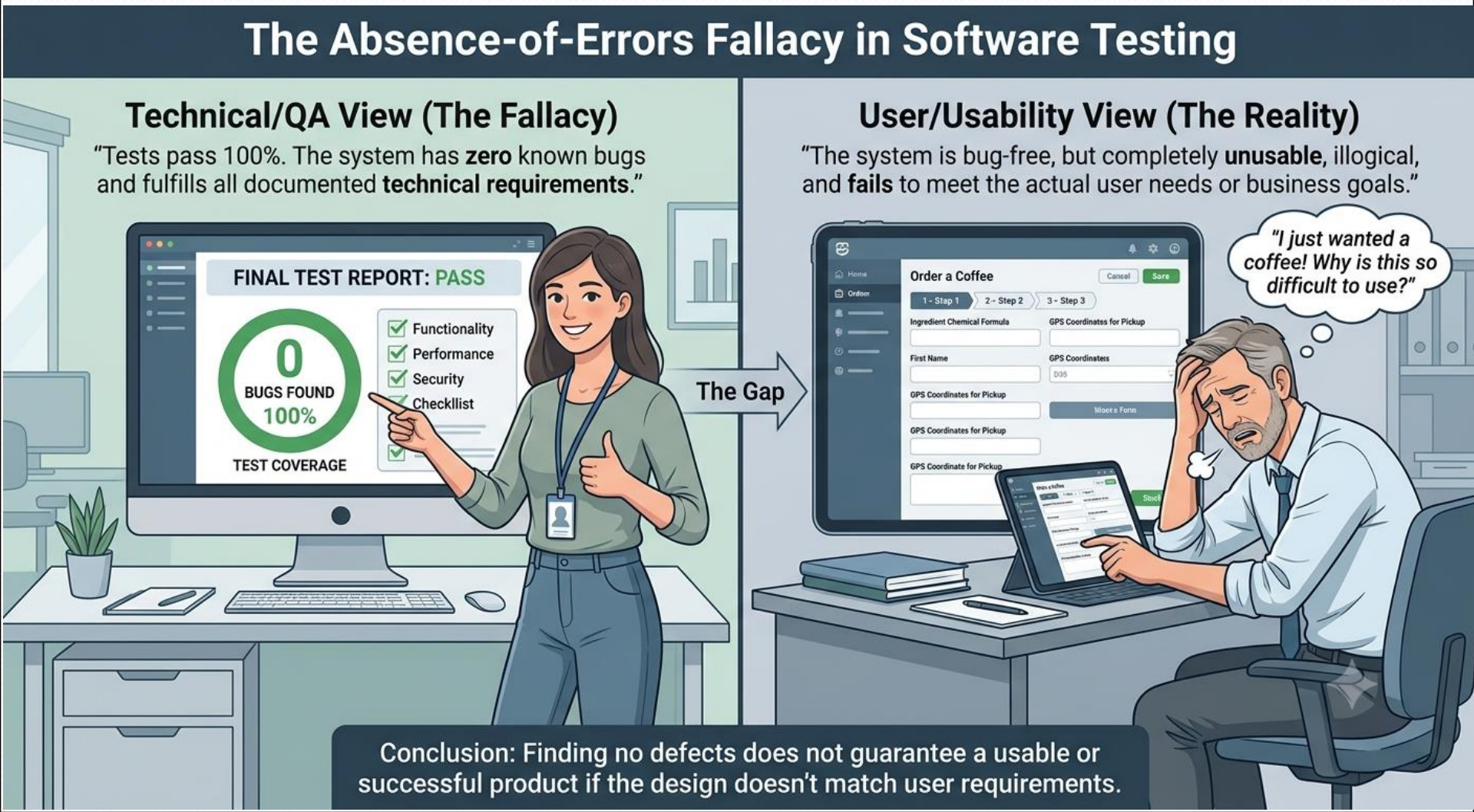
Testing Principles:

- **Principle 6**:Testing is Context Dependent.

Example : Testing an **ATM machine** requires extreme security, performance, and hardware integration testing. Testing a **mobile game** requires UI/UX, battery drain, and touch gesture testing. The *approach* and *techniques* change completely based on the application type.

Testing Principles:

- **Principle 7:** Absence-of-Errors Fallacy.



Testing Principles:

- **Principle 7:** Absence-of-Errors Fallacy.

Example : The software has 0 known bugs, 100% test coverage, and is incredibly fast. BUT, it does not allow users to reset their forgotten password. It is completely useless. Fixing "wrong" bugs doesn't matter if the core user journey fails.

Test Case vs Test Plan:

Concept	1-Line Simple Definition
Test Plan	**The MASTER STRATEGY** – *What* to test, *Who* will test, *When* will it end. (The "Bird's Eye View").
Test Case	**The STEP-BY-STEP RECIPE** – Exactly *how* to test one single thing, what data to enter, and what to expect. (The "Ground Level Action").

Test Plan (The Master Strategy):

- **Project:** You are testing the "**Add to Cart & Checkout**" feature for an e-commerce app like **Flipkart/Amazon** for the upcoming Diwali sale.

Here is what goes inside the **Test Plan Document**:

- **Scope (What is IN?):** Only testing "Add to Cart", "Apply Coupon", and "Payment via UPI".
  (Scope OUT: We are NOT testing "Product Search" or "Return/Refund" – that's another team).
- **Schedule (When?):** Testing will start on Monday 9:00 AM and must end by Wednesday 6:00 PM (so developers can fix bugs before the sale).
- **Resources (Who?):** 2 Testers (Rahul and Priya), using 2 Windows Laptops, 1 iPhone, and 1 Android phone.

Test Plan (The Master Strategy):

- **Project:** You are testing the "**Add to Cart & Checkout**" feature for an e-commerce app like **Flipkart/Amazon** for the upcoming Diwali sale.

- **Entry Criteria (When to START?):** We will start testing ONLY when the developers give us the "Build v2.1" and the login page works properly (Smoke Test passed).
- **Exit Criteria (When to STOP?):** We will stop testing when 95% of test cases are PASS, and there are ZERO "Critical" bugs that crash the app.
- **Risk Plan (What if?):** What if the payment gateway (like PhonePe) is down during testing? We will use a "Mock" fake UPI to test instead.

Test Case (Step by Step Strategy):

- If the Test Plan is the Master Strategy, the **Test Case** is the tiny, exact instruction for checking **ONE scenario** (e.g., "Successful Payment").

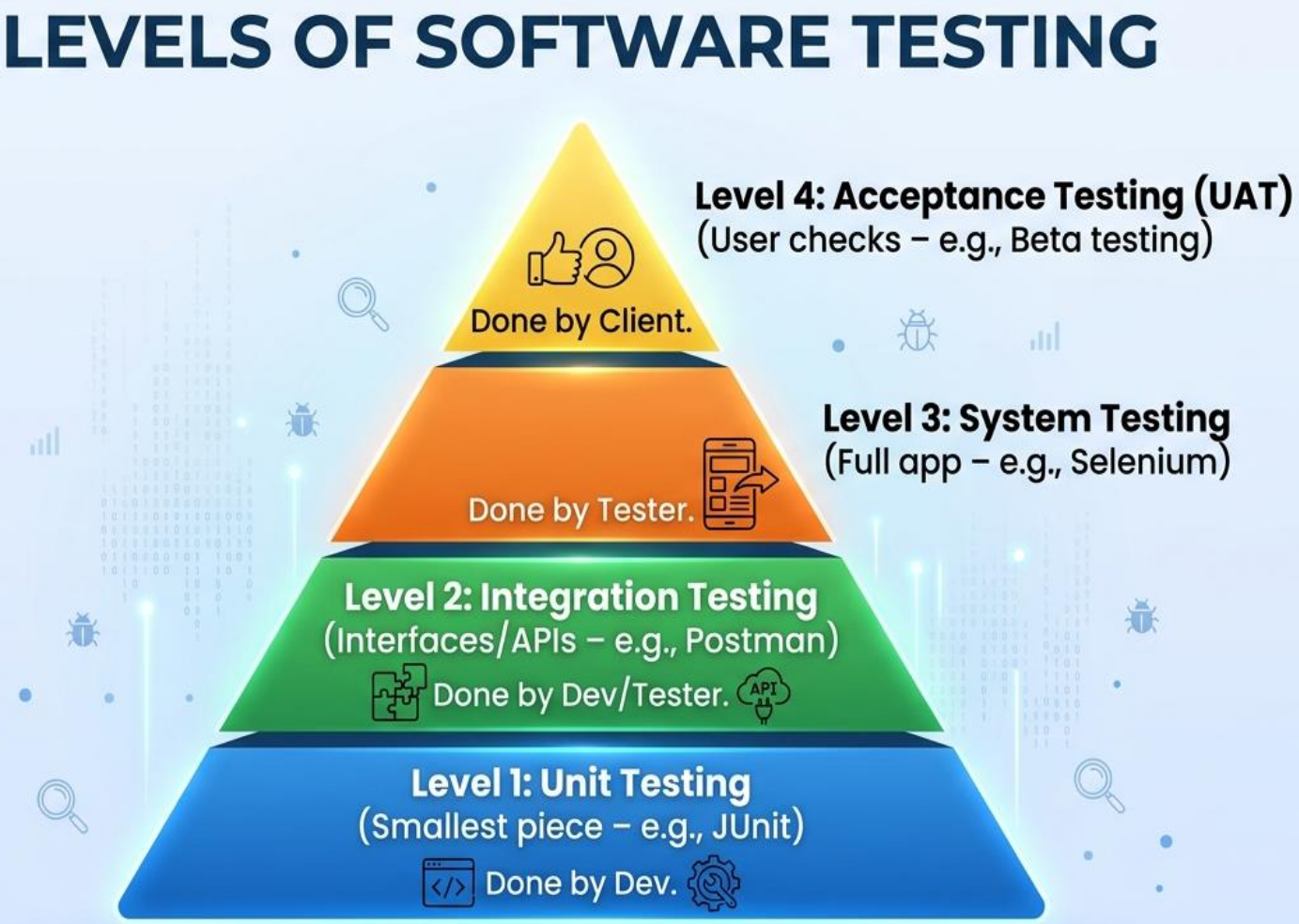
Test Case (Step by Step Strategy):

Field	Actual Text Filled by Tester
TC_ID	TC_CHECKOUT_101
Title	Verify successful order placement using Valid UPI.
Pre-condition	User must be logged in. At least 1 item must already be in the cart.
Test Data	Product = "Nike Shoes", UPI ID = "legend@paytm"
Steps to Execute	1. Open the App. 2. Click on the "Cart" icon. 3. Click the "Place Order" button. 4. Select "Pay via UPI" option. 5. Enter UPI ID: "legend@paytm". 6. Click "Pay Now".
Expected Result	The app should show a **"Order Confirmed"** message with an Order ID (e.g., #ORD12345). The balance should be deducted.
Actual Result	(Tester writes after execution): Order Confirmed with ID #ORD12345 shown.
Final Status	**PASS** (because Expected = Actual)

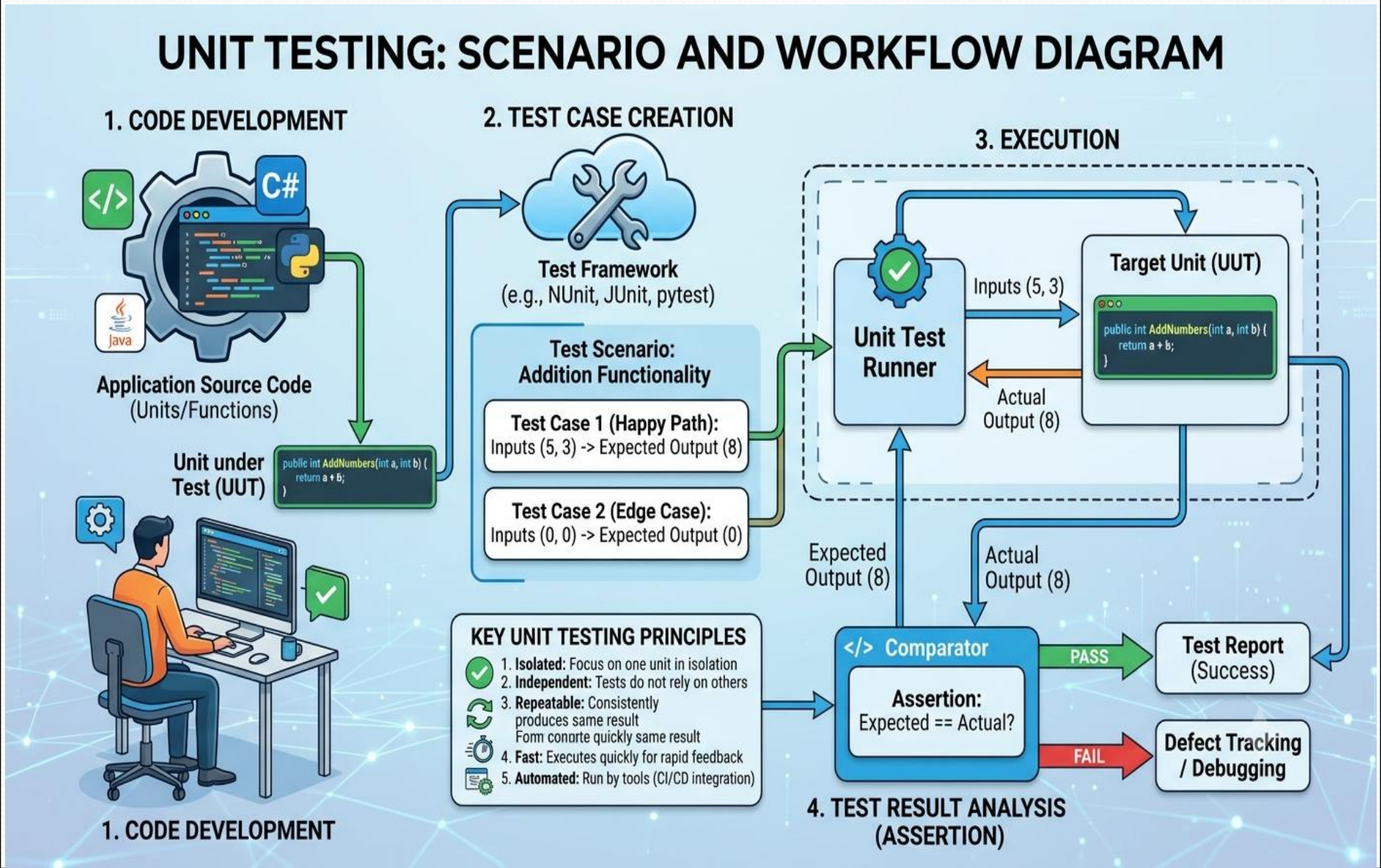
Real Time Comparison Test Case Vs Test Plan:

Question	Test Plan (Answered by Manager)	Test Case (Answered by Tester)
What is it?	A 10-page Word/Excel document with timelines.	A single row in an Excel sheet or a JIRA ticket.
What happens if the "Payment" fails?	The Plan says: "We will stop testing and report a Critical bug to the Dev team."	The Test Case says: "Enter UPI PIN -> If error shows, mark status as FAIL."
How many of these exist?	Exactly 1 Master Plan for the entire Diwali release.	500 to 1000 Test Cases to check every tiny thing (Login, Search, Cart, Payment, Logout).
Who creates it?	The Test Lead / Manager (because they know the big picture).	The SDET / Junior Tester (because they know the click-by-click details).

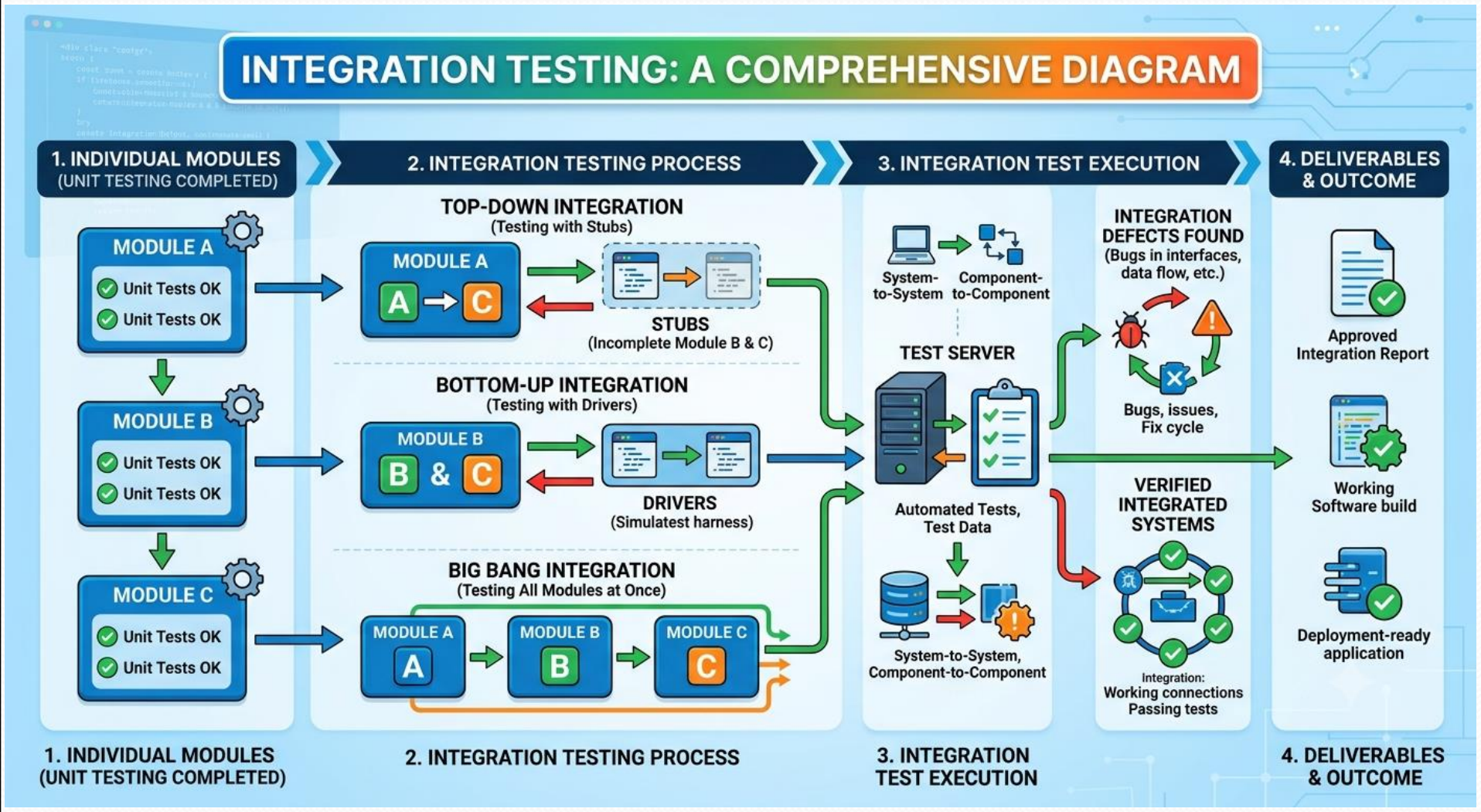
Levels of Testing:



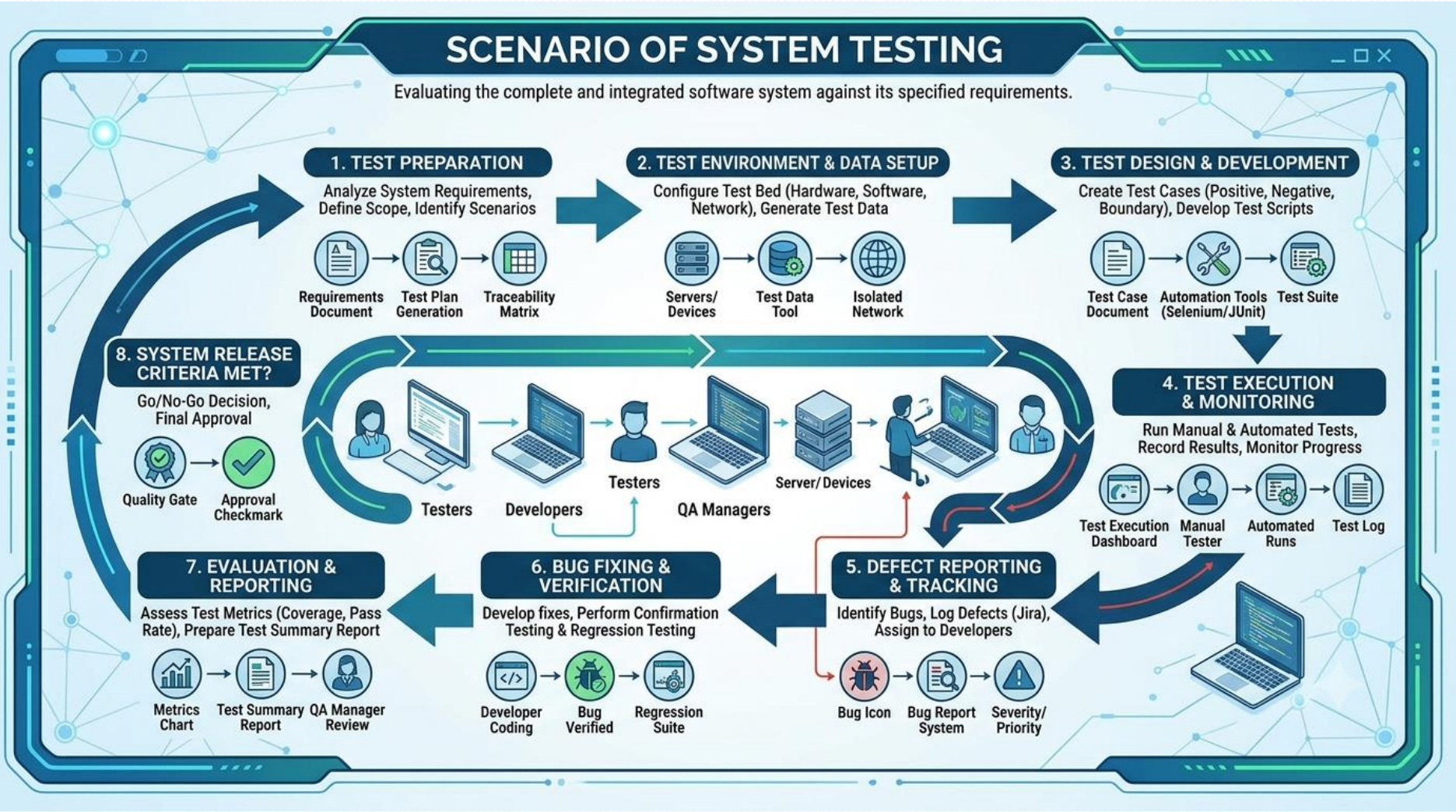
Unit Testing: (Developers Task)



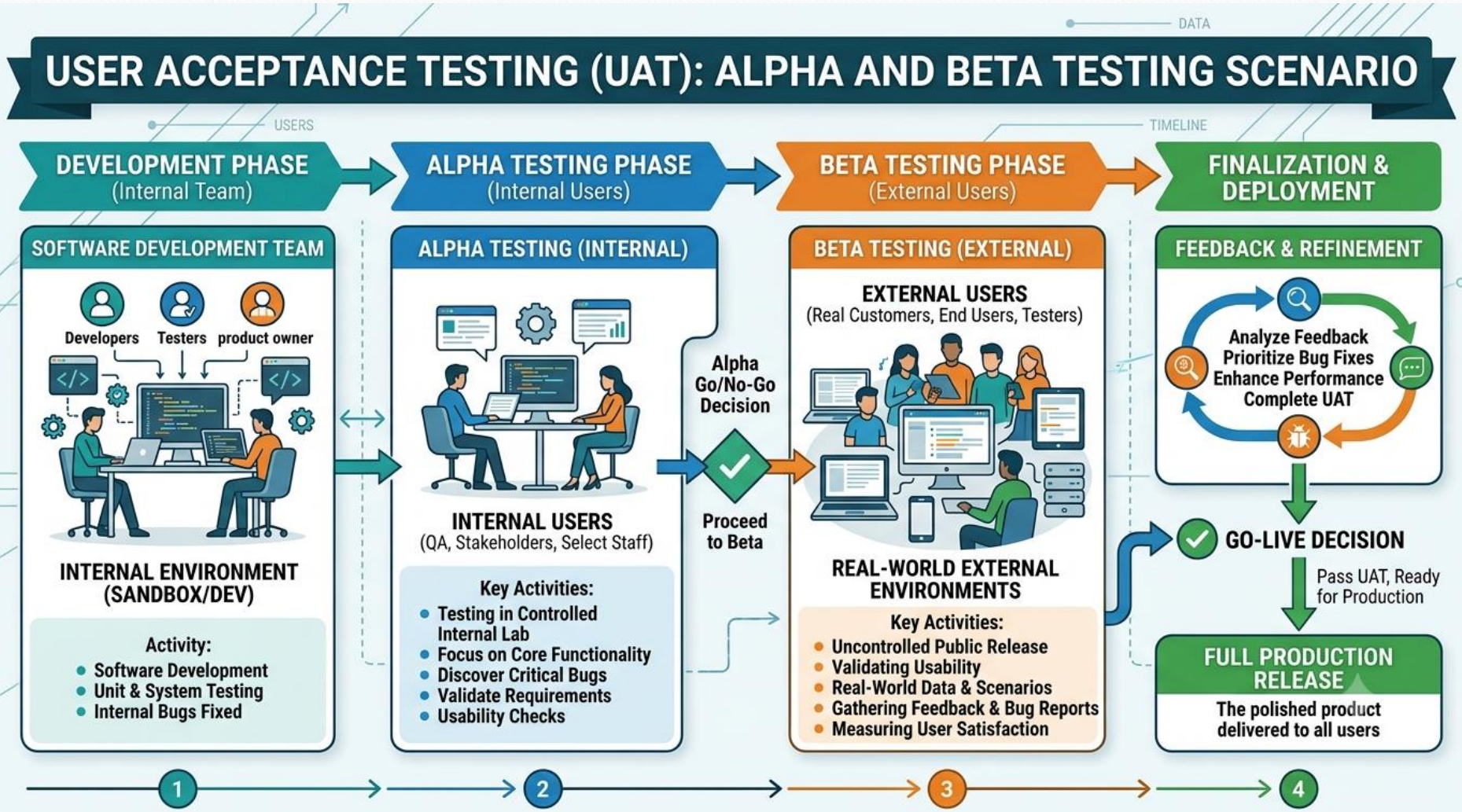
Integration Testing: (Testers Task)



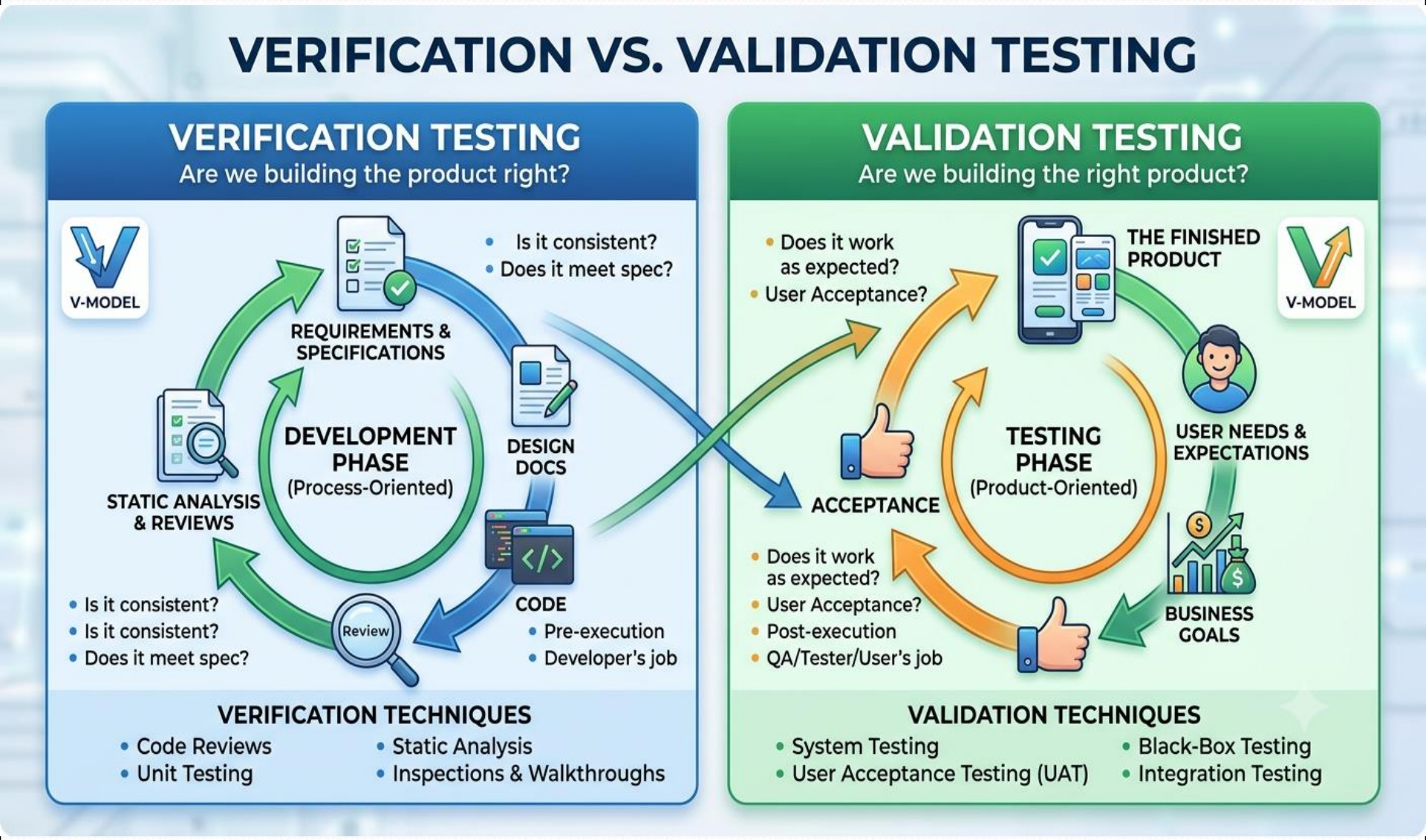
System Testing: (Testers Task)



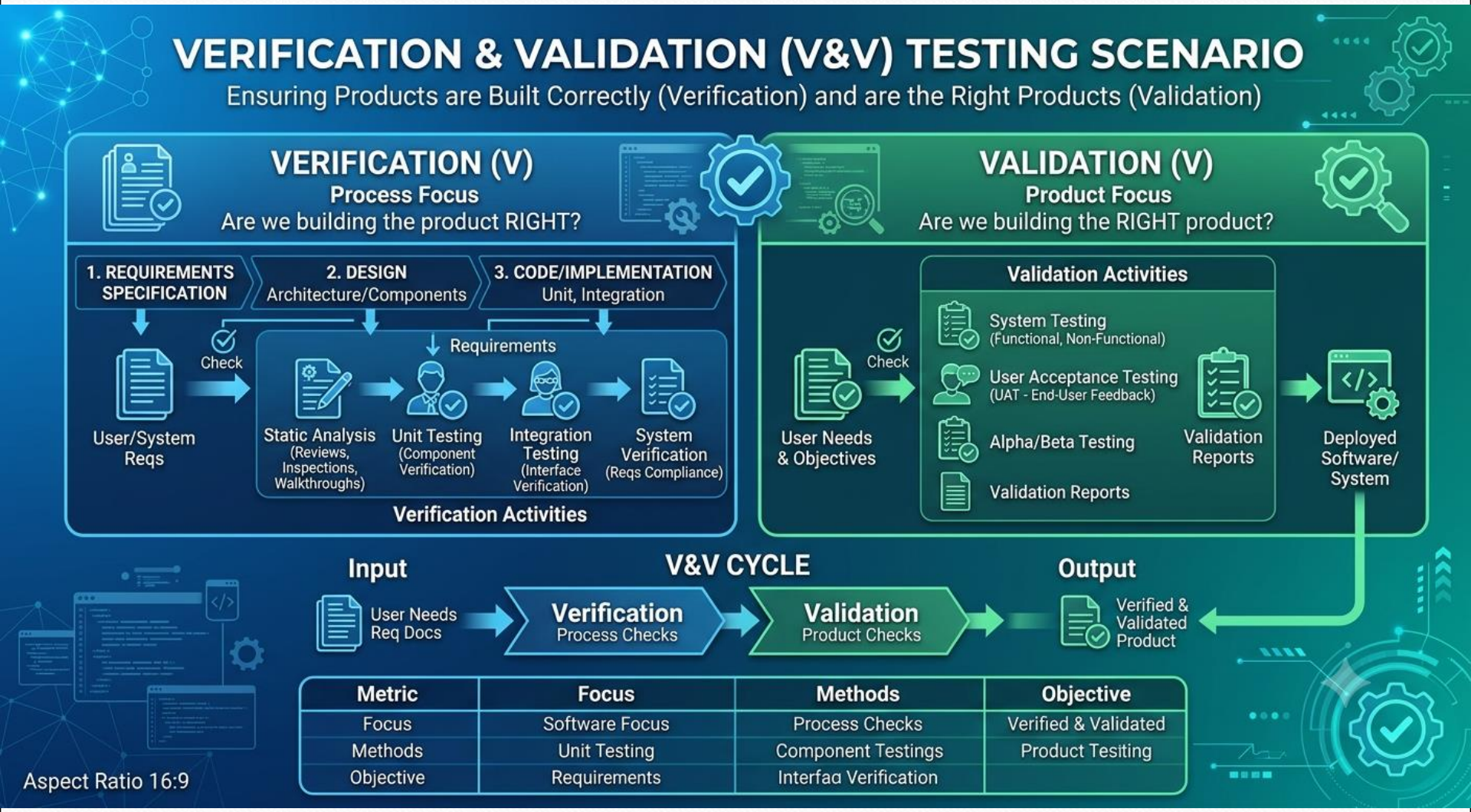
User Acceptance Testing: (UAT-Users Task)



Verification v/s Validation Testing:



Verification & Validation Testing:



Metric	Focus	Methods	Objective
Focus	Software Focus	Process Checks	Verified & Validated
Methods	Unit Testing	Component Testings	Product Tesiting
Objective	Requirements	Interfaa Verification	

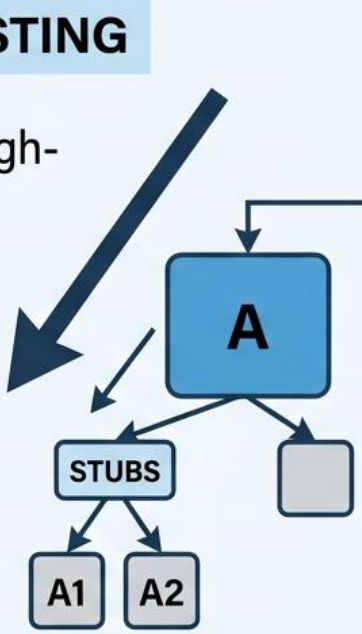
Testing Strategies (Integration Approach):

TOP-DOWN TESTING

Top-Down: Test high-level modules first.

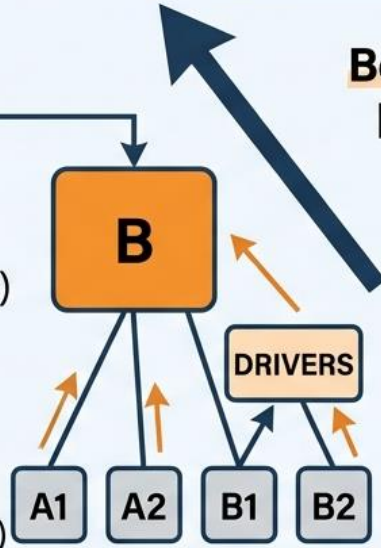
Used ton when lower modules - not ready are not ready.

(**Need:** Stubs - dummy lower modules)



HIGH-LEVEL MODULES
(e.g., Module A, B)

LOW-LEVEL MODULES
(e.g., A1, A2, B1, B2)



BOTTOM-UP TESTING

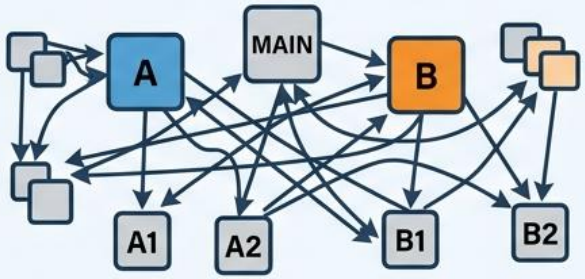
Bottom-Up: Test low-level modules first.

Used to ad to controllol tests in the absence of complete complete higher-level.

(**Need:** Drivers - dummy higher modules)

BIG-BANG TESTING

Big-Bang: Test everything together at the end. (High Risk!)

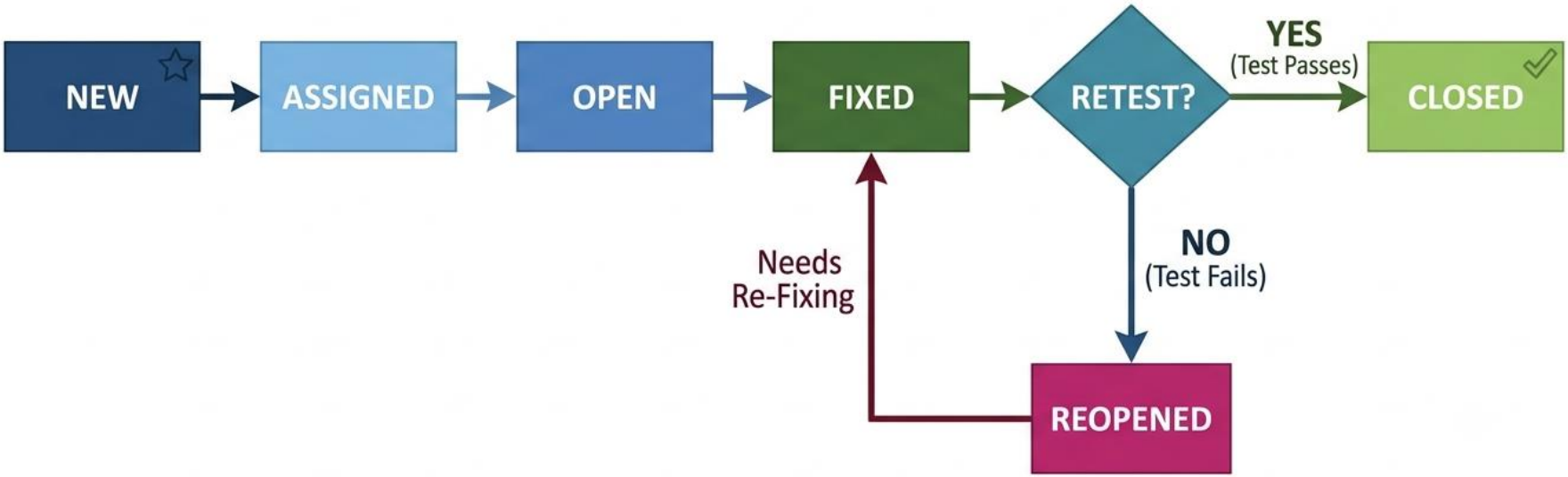


Big-Bang: Test everything together at the end.

Defect Life Cycle (The Journey of a Bug)

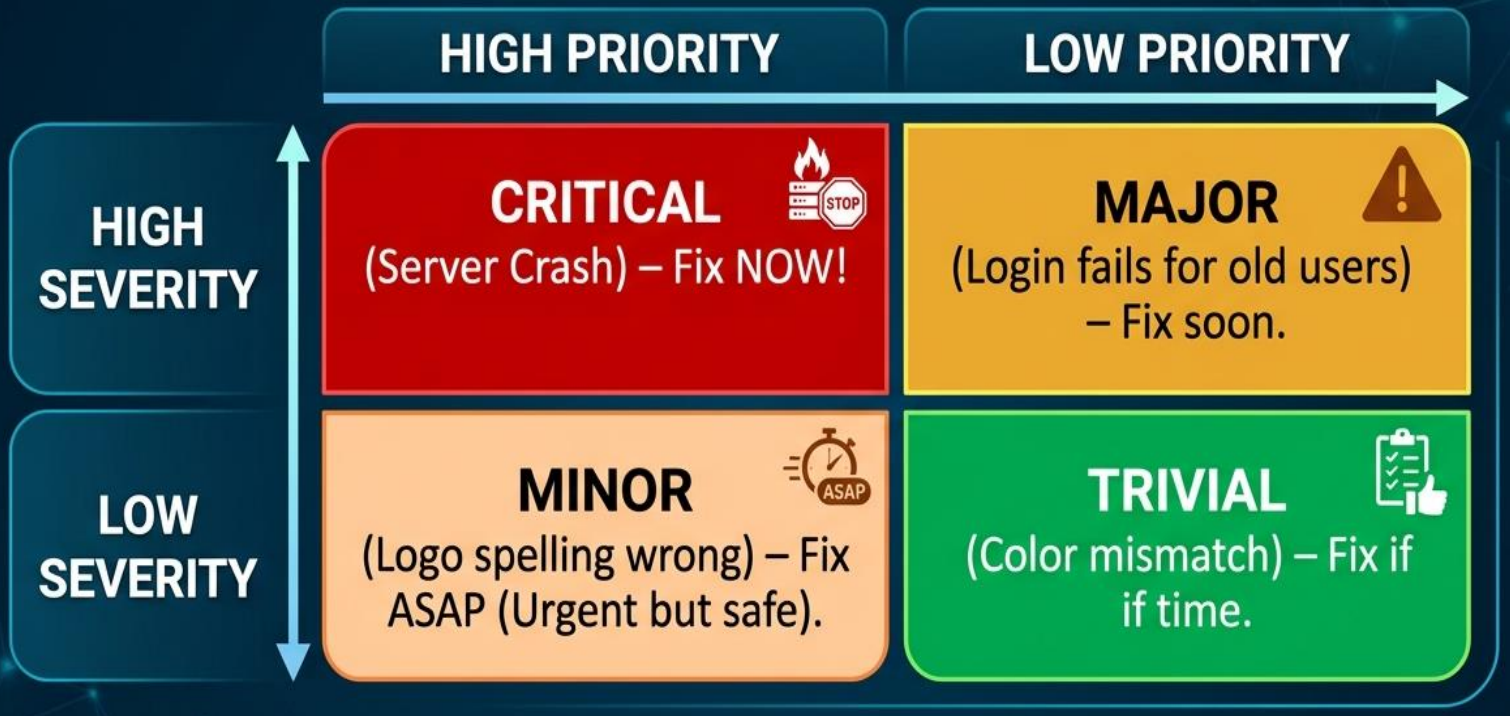
Title: ISSUE TRACKING WORKFLOW DIAGRAM

Prompt: New ➡ Assigned ➡ Open ➡ Fixed ➡ Retest ➡ Closed
Condition: If Retest fails: **Reopened** (goes back to Fixing)



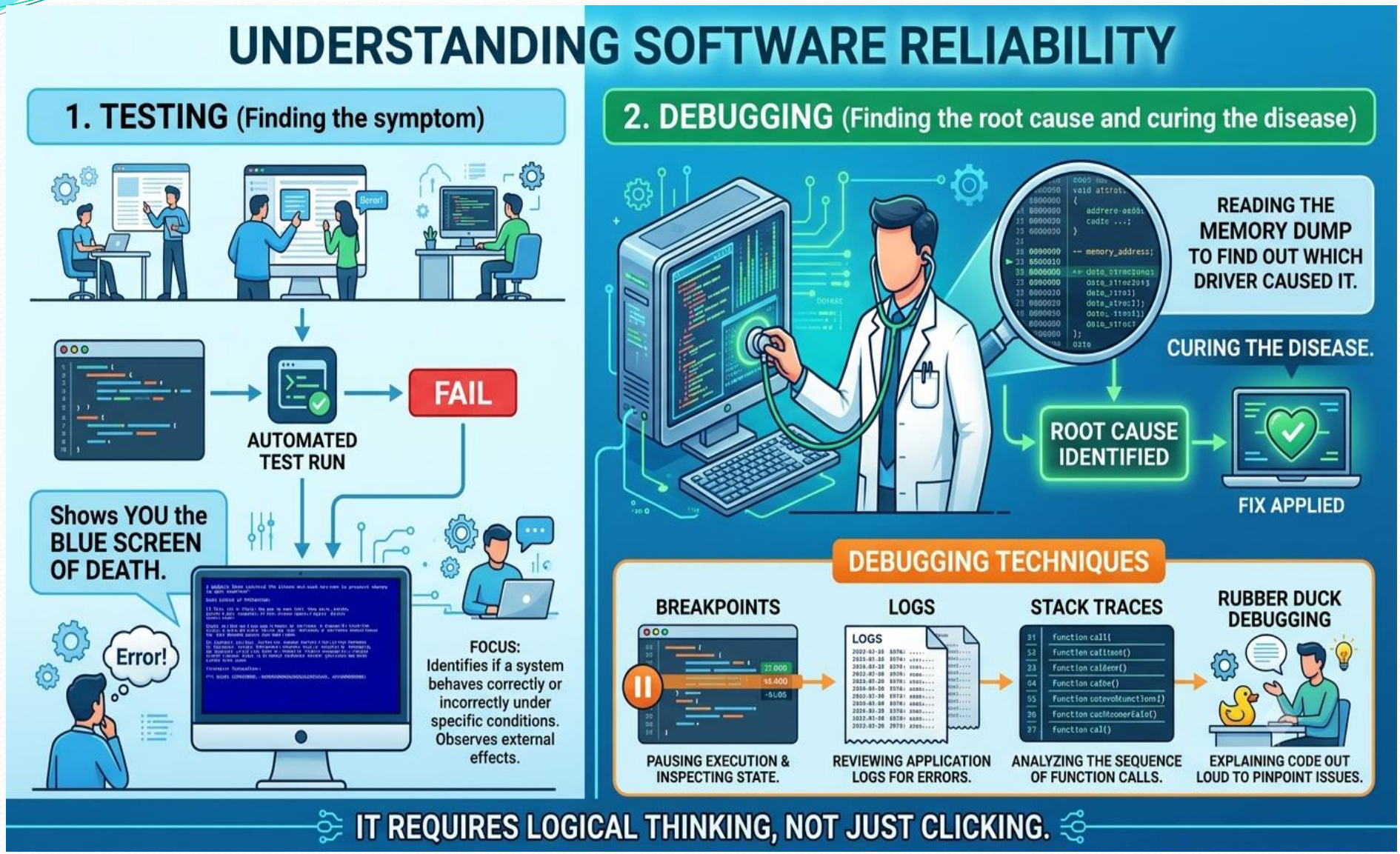
Bug Reporting - Severity vs. Priority

SLIDE TEXT (USE A 2X2 MATRIX)

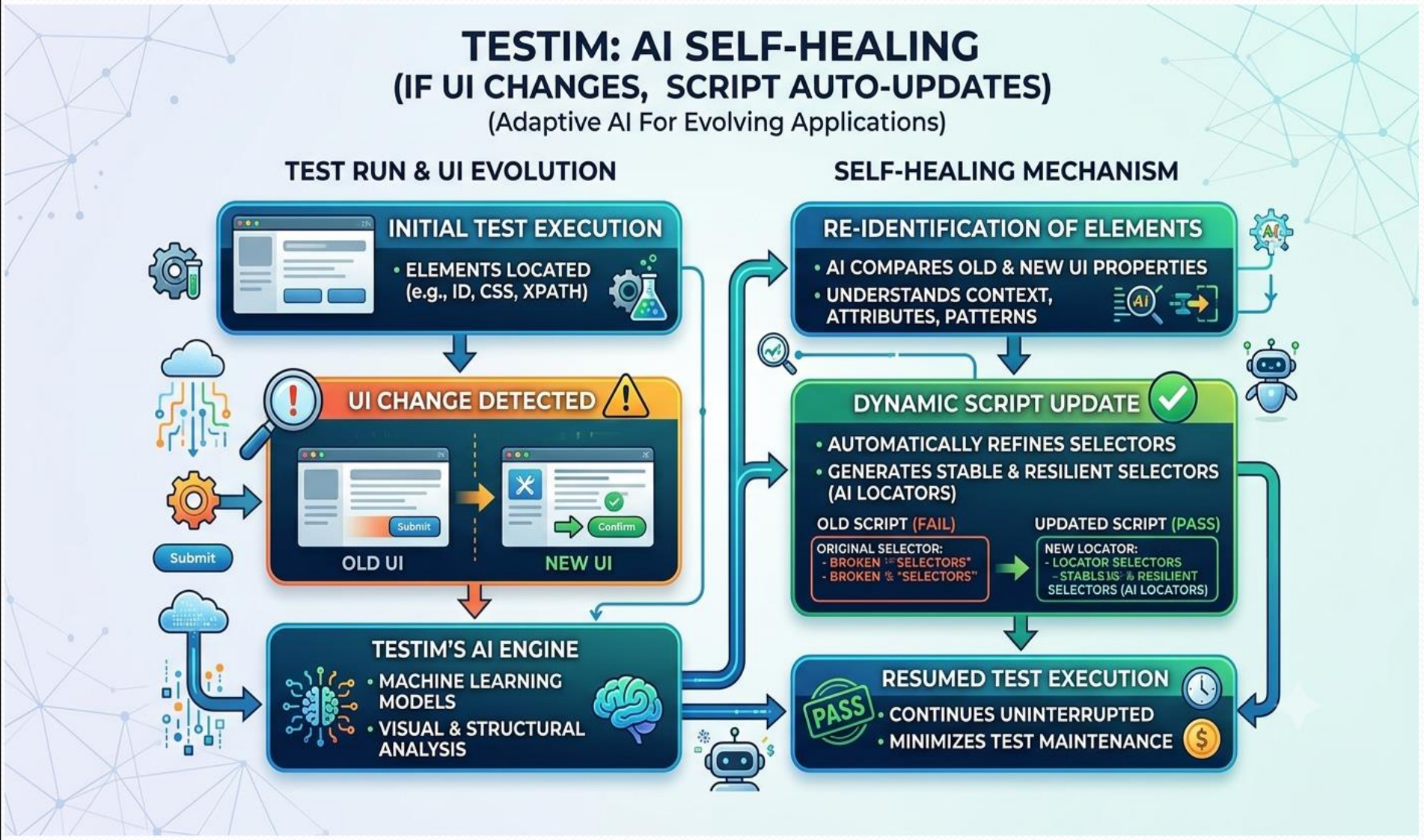


SPEAKER NOTE: Severity = Impact on system (Technical). Priority = Impact on Business (Urgency). A spelling mistake on the homepage is Low Severity but HIGH Priority because the CEO will see it!

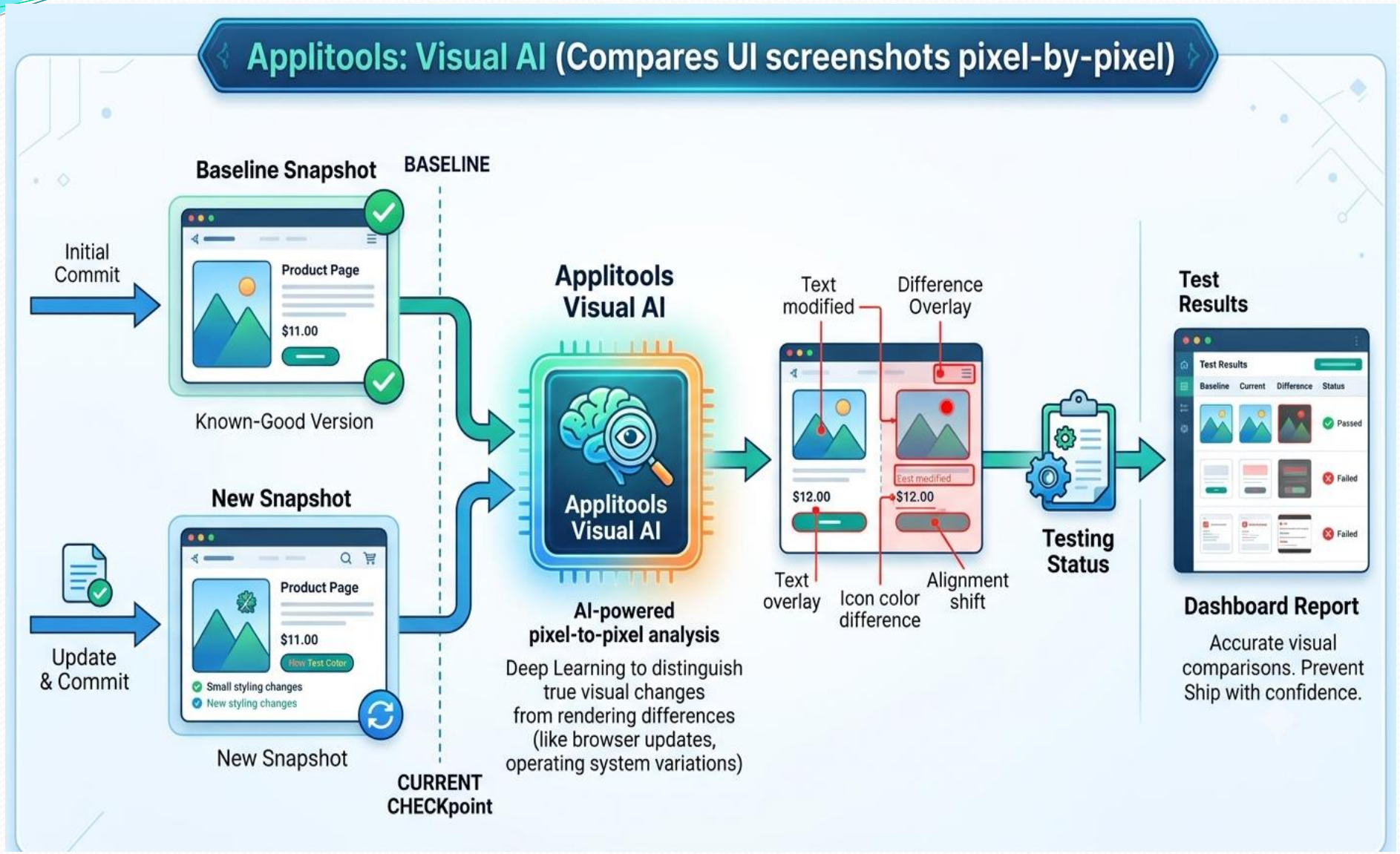
Debugging (The Art of Fixing)



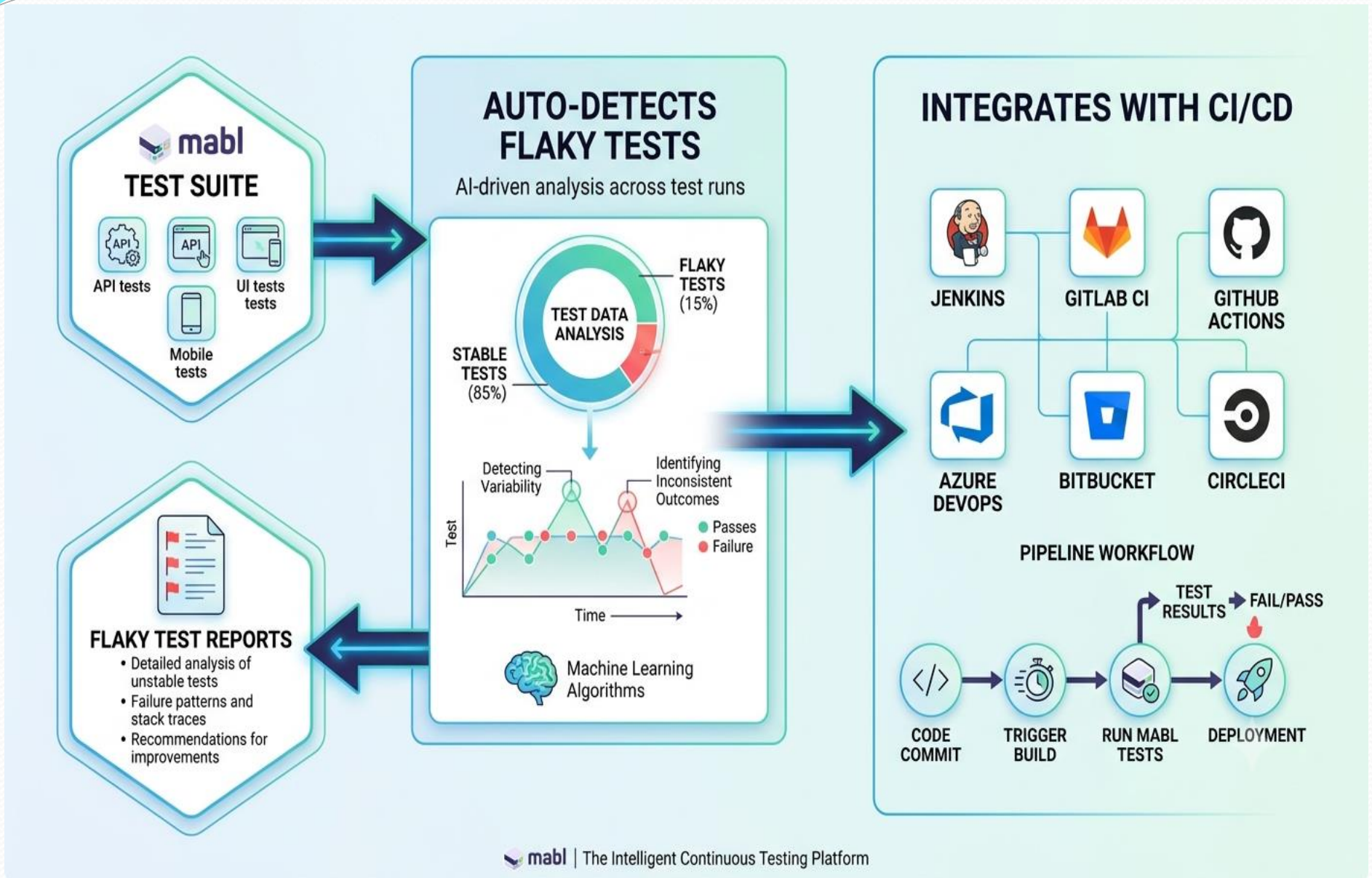
AI-Powered Tools (The Future is Here)



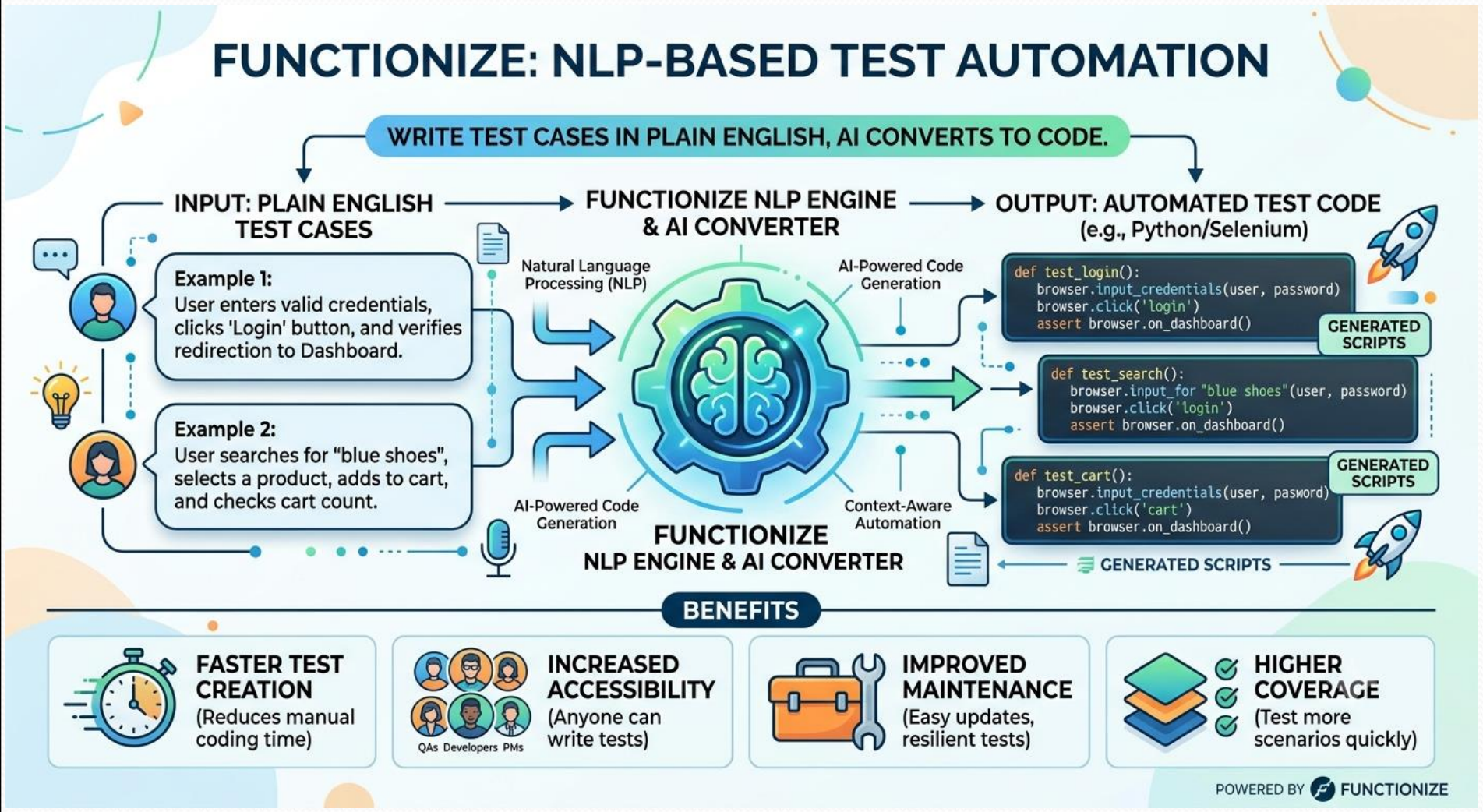
AI-Powered Tools (The Future is Here)



AI-Powered Tools (The Future is Here)



AI-Powered Tools (The Future is Here)



AI Benefits, Limitations & Future Trends

